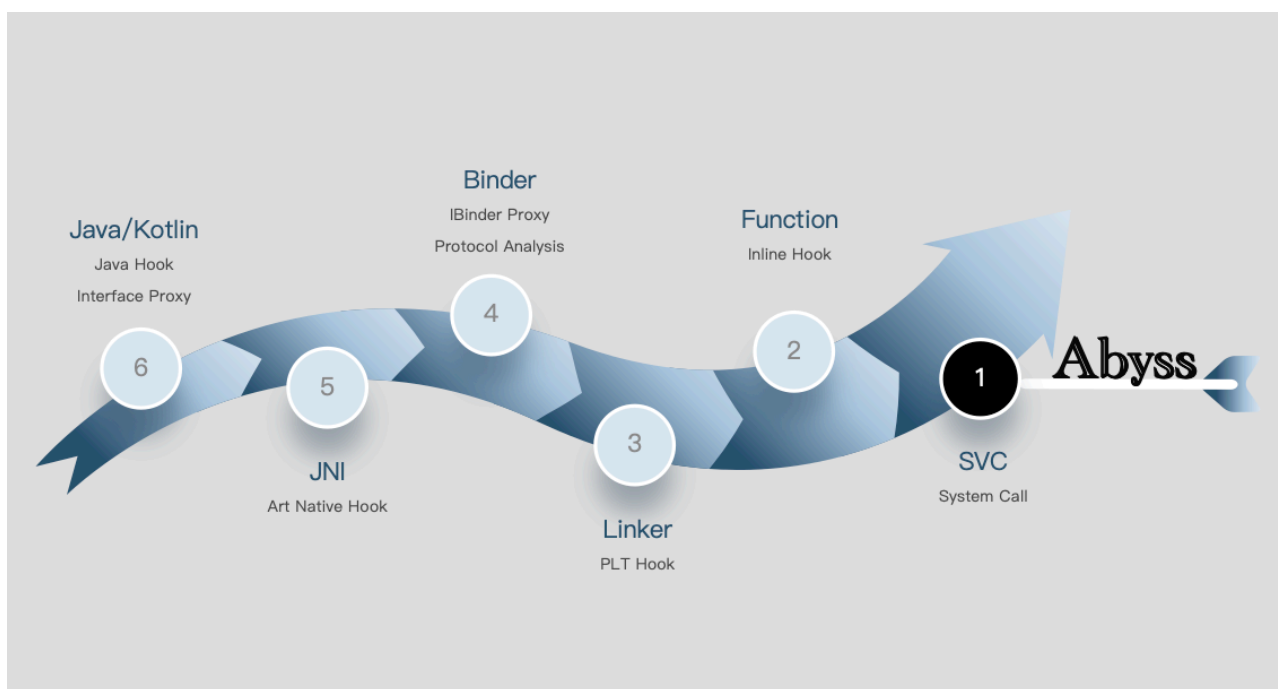


[← 返回](#)

• 阅读需 7 分钟

【Abyss】Android平台应用级系统调用拦截框架

Android 平台从上到下，无需ROOT/解锁/刷机，应用级拦截框架的最后一环 —— SVC 系统调用拦截。

[👉 Github 👈](#)

由于我们虚拟化产品的需求，需要支持在普通的 Android 手机运行。我们需要搭建覆盖应用从上到下各层的应用级拦截框架，而 Abyss 作为系统 SVC 指令的调用拦截，是我们最底层的终极方案。

01. 说明

tracee: 被 `ptrace` 附加的进程，通常为目标应用进程。

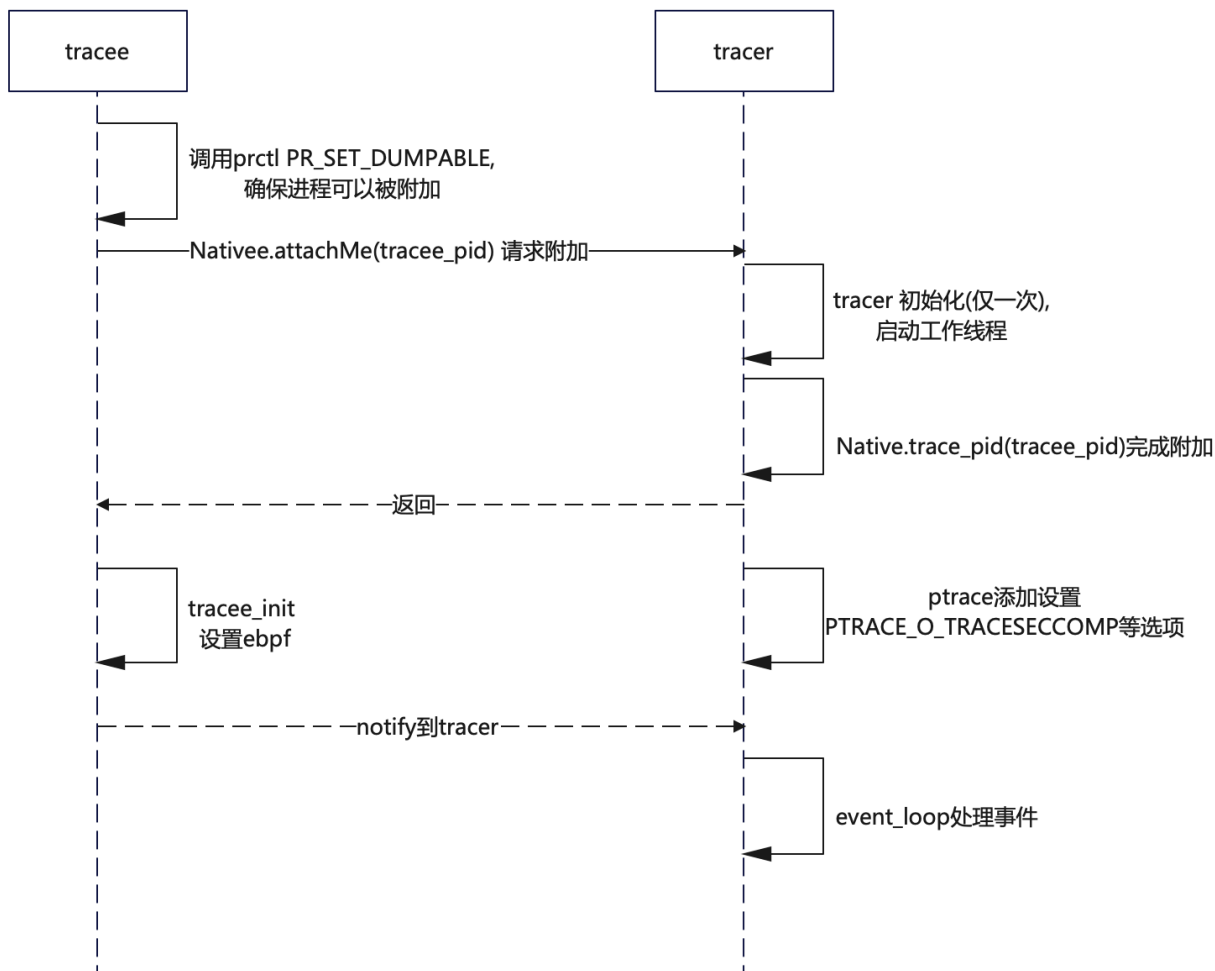
tracer: 用来 `ptrace` 其他进程的进程，在该进程里处理系统调用。

本框架利用 `Android` 的 `Provider` 组件启动拦截处理的服务进程，进程启动后创建独立的一个线程循环处理所有拦截的系统调用回调。由于本工程只是演示方案的可行性并打印日志，所以业务逻辑处理比较简单，可以根据需要的自行扩展。

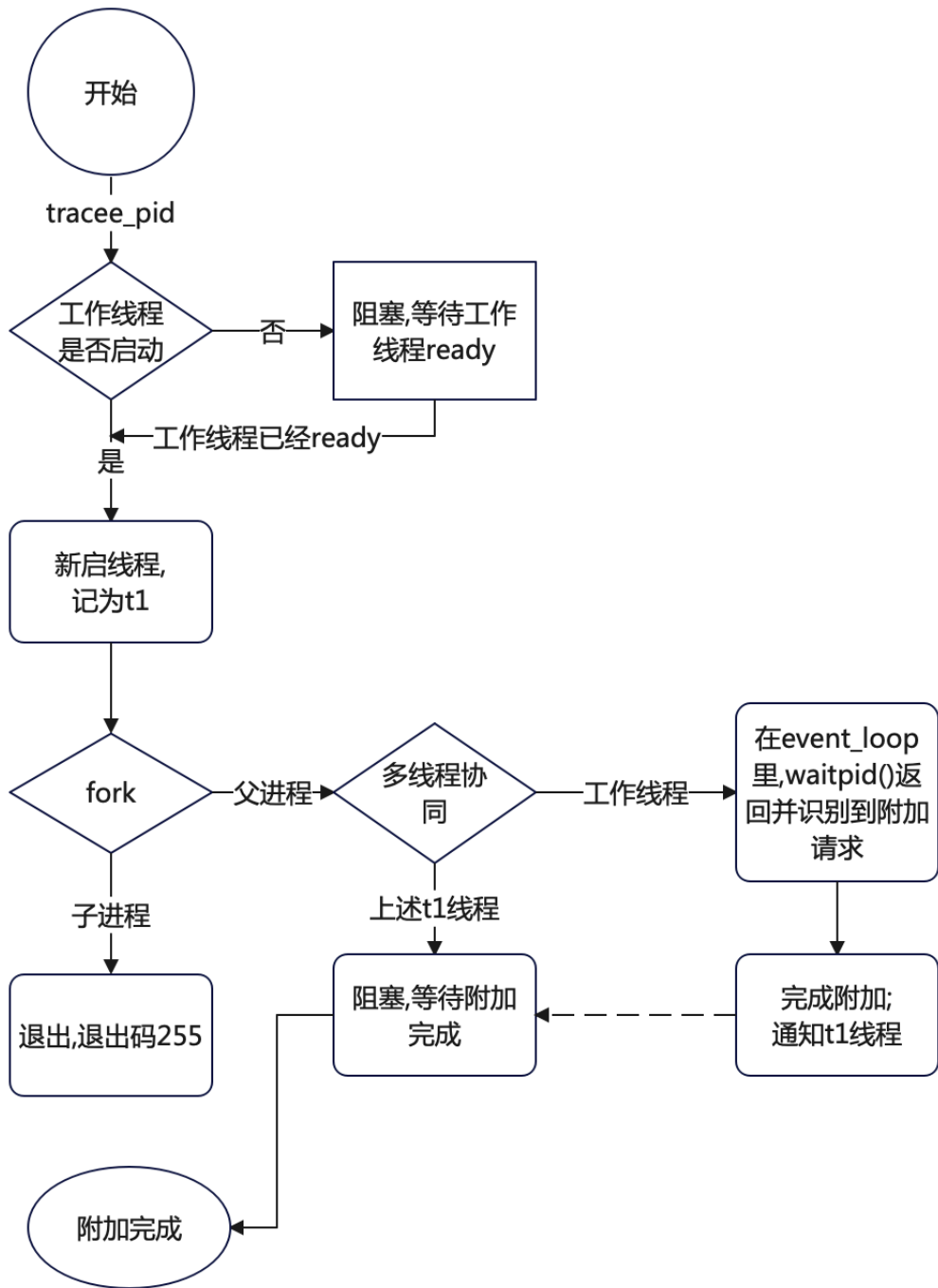
若要接入具体业务，可能需要改成多线程的方式进行处理，提升稳定性。不过我们实测多线程切换也有一定损耗，性能提升有限，但确实稳定性有提升，防止某个处理耗时导致应用所有进程阻塞。

02. 处理流程

应用进程 `tracee` 被附加流程如下：



`tracer` 过程如下：



说明： 使用 `fork()` 的目的是为了让工作线程去附加。 `ptrace` 有严格的限制，只有执行附加 `attach` 的线程才有权限操作对应 `tracee` 的寄存器。

03. 系统调用处理

03.01 忽略库机制

由于业务的需要，为了提升性能，我们需要忽略某些库中的系统调用，如： `libc.so` 。

在 `find_libc_exec_maps()` 中找到 `libc.so` 可执行代码在 `maps` 中的内存地址区间，需要处理的系统调用：

```
//enable_syscall_filtering()
FilteredSysnum internal_sysnums[] = {
    { PR_ptrace,          FILTER_SYSEXIT },
    { PR_wait4,          FILTER_SYSEXIT },
    { PR_waitpid,         FILTER_SYSEXIT },
    { PR_execve,          FILTER_SYSEXIT },
    { PR_execveat,        FILTER_SYSEXIT },
    { PR_readlinkat,      FILTER_SYSEXIT}, //暂时没有处理
};
```

`set_seccomp_filters` 针对不同的 `arch`，设置系统调用的 `ebpf`。不同架构的 `ebpf` 语句会填充到一起，`ebpf` 的组成伪代码如下：

```
for (每一种架构) {
    start_arch_section;
    for (每一个当前架构下的系统调用)
        add_trace_syscall;
    end_arch_section;
}
finalize_program_filter;

start_arch_section; // 架构相关处理的ebpf, 包括libc筛选的语句
add_trace_syscall; // 增加匹配要处理系统调用的ebpf语句
end_arch_section; // 尾部的ebpf语句(语句含义: 匹配到系统调用则返回)
finalize_program_filter; // 最后面的ebpf语句, 杀死其他异常情况下的线程
```

最终，调用如下语句，设置 `ebpf`。

```
status = prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &program);
```

03.02 PR_ptrace

因为一个 `tracee` 只能有一个 `tracer`，所以需要处理该系统调用，在应用本身使用了 `ptrace` 的时候进行仿真。

系统调用进入前，将系统调用替换为 `PR_void`，不做真正的 `ptrace`，后续仿真。

退出系统调用后，针对 `ptrace` 的仿真。针对请求是 `PTRACE_ATTACH`、`PTRACE_TRACEME` 等做各种不同的处理。同时也处理 `PTRACE_SYSCALL`、`PTRACE_CONT`、`PTRACE_SETOPTIONS`、`PTRACE_GETEVENTMSG` 等各种 `ptrace` 操作。

`ptrace` 有各种各样的请求，完整的处理逻辑比较复杂（我们还在消化中）。

03.03 PR_wait4、PR_waitpid

配合 `PR_ptrace` 使用，如果当前的 `tracee` 不是一个 `tracer`，则不处理直接透传给系统。或者 `wait` 的第一个参数不为 `-1`，则去集合里找看等待的这个线程是否存在并且是否是当前处理线程的 `tracee`，如果不是，则不处理直接透传给系统。

处理的逻辑如下：

系统调用进入前，将系统调用替换为 `PR_void`，不实际传给内核。

退出系统调用后，仿真 `tracer` 里 `wait` 的处理逻辑。主要为基于当前处理的这个 `tracer`（代码里定义为 `ptracer`），去遍历它的 `tracee`，看是否有事件需要被处理，如有，则填充好寄存器，唤醒当前正在被处理的这个 `tracer`。

03.04 PR_execve、PR_execveat

主要是在 `USE_LOADER_EXE` 开启时，将 `native` 程序替换为使用一个固定的 `loader` 来加载程序。

03.05 拦截日志

```
E INTERCEPT/SYS: vpid 2: got event 7057f
E INTERCEPT: vpid 2,secomp_enabled 0,
E INTERCEPT/SYS: (null) info: vpid 2: sysenter start: openat(0xffffffff9c,
0xb4000073c72fcd60, 0x0, 0x0, 0xb4000073c72fcd88, 0xb4000073c72fcde8) = 0xffffffff9c
[0x7367d45e80, 0]
E INTERCEPT/SYS: vpid 2: open path:/system/fonts/NotoSansMalayalamUI-VF.ttf
E INTERCEPT/SYS: syscall_number:216
E INTERCEPT/SYS: vpid 2,openat: /system/fonts/NotoSansMalayalamUI-VF.ttf
E INTERCEPT/SYS: (null) info: vpid 2: sysenter end: openat(0xffffffff9c,
0xb4000073c72fcd60, 0x0, 0x0, 0xb4000073c72fcd88, 0xb4000073c72fcde8) = 0xffffffff9c
[0x7367d45e80, 0]
E INTERCEPT/SYS: vpid 2: open path:/system/fonts/NotoSansMalayalamUI-VF.ttf
E INTERCEPT/SYS: (null) info: vpid 2: restarted using 7, signal 0, tracee pid
32222,app_pid 32162

E/INTERCEPT/SYS: (null) info: vpid 3: sysenter start: close(0x90, 0x0, 0x7492d0d088,
0x6, 0x73b7b82860, 0x73b7b82880) = 0x90 [0x73633faae0, 0]
E/INTERCEPT/SYS: syscall_number:41
E/INTERCEPT/SYSW: noting to do,sn:41
E/INTERCEPT/SYS: (null) info: vpid 3: sysenter end: close(0x90, 0x0, 0x7492d0d088,
0x6, 0x73b7b82860, 0x73b7b82880) = 0x90 [0x73633faae0, 0]
E/INTERCEPT/SYS: (null) info: vpid 3: restarted using 7, signal 0, tracee pid
32223,app_pid 32162
E/INTERCEPT/SYS: vpid 3: got event 7057f
```

04. 附

额外模块：

由于本框架会在原应用中增加一个处理进程，并且会 `trace` 到应用进程中，因此在实际使用时，还需要对新增进程和 `trace` 痕迹进行隐藏，防止与应用检测模块冲突，支持完整的应用自身 `trace` 调用的仿真。

这是附加的应用对抗模块，后面会作为单独文章分享给大家。

参考项目：

<https://github.com/proot-me/proot>

<https://github.com/termux/proot>

← [上一页](#)

[FMOD调音框架之内存级控制](#)

[下一页](#) →

[凡人歌：平凡公司的技术面试](#)